

Cours d'introduction à la Data Science

Chargé du cours : Jules DEGILA, PhD, MCF
Assistante : Aurélie KPOZE, PhD

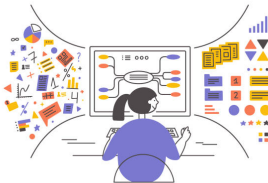
jules.degila@imsp-uac.org, satou.kpoze@imsp-uac.org

23 Janvier 2026



Chapitre 5

Fondamentaux du Deep Learning



Concepts clés, architectures et méthodes d'entraînement des réseaux de neurones profonds.

Plan du Chapitre 5

- ➊ Rappels sur le Machine Learning et limites des modèles classiques
- ➋ Neurone artificiel et perceptron
- ➌ Réseaux de neurones multicouches (MLP)
- ➍ Fonctions d'activation
- ➎ Fonction de coût et optimisation
- ➏ Rétropropagation du gradient (Backpropagation)
- ➐ Entraînement d'un réseau de neurones
- ➑ Problèmes courants : surapprentissage, underfitting, régularisation
- ➒ Introduction aux architectures profondes (CNN, RNN)

À la fin du chapitre, vous serez capables de :

- Comprendre les bases du Deep Learning
- Modéliser un neurone artificiel
- Expliquer l'apprentissage par rétropropagation
- Identifier les problèmes et limites des réseaux de neurones
- Découvrir les architectures profondes modernes

Quand un réseau devient très profond :

- L'apprentissage devient difficile
- Le gradient disparaît (vanishing gradient)
- Le modèle n'apprend plus correctement

Donc : plus profond n'est pas toujours meilleur.

ResNet = Residual Network (2015)

Idée simple :

- Au lieu d'apprendre directement la transformation
- Le réseau apprend la **différence (résidu)**

Formule :

- $y = x + F(x)$ au lieu de : $y = F(x)$

Pourquoi c'est puissant ?

Grâce aux connexions résiduelles :

- Le gradient circule mieux
- Les réseaux très profonds deviennent entraînaables
- Meilleure performance

Sans ResNet :

- Le réseau doit tout apprendre

Avec ResNet :

- Si une couche n'est pas utile → elle peut "ne rien changer"
- L'information originale est conservée

Donc : apprentissage plus stable.

Reconnaissance d'image :

- CNN simple : 5-10 couches
- ResNet : 50, 101, 152 couches

ResNet gagne ImageNet car :

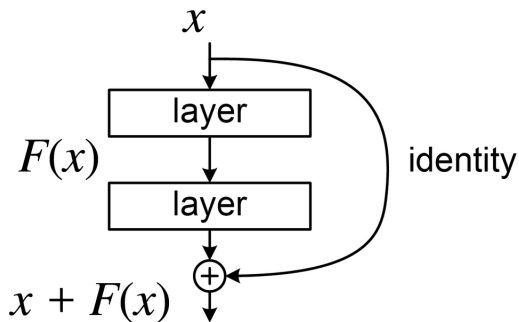
- Réseau très profond
- Apprentissage stable

Où utilise-t-on ResNet ?

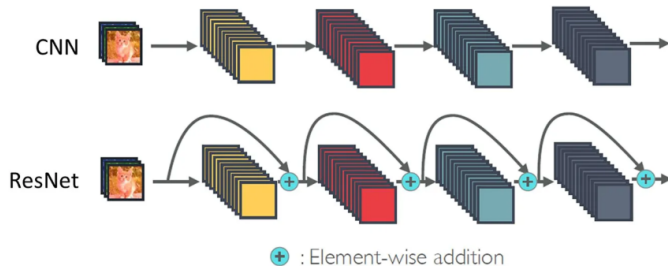
- Vision par ordinateur
- Détection d'objets
- Médecine (radiologie)
- Voitures autonomes
- Reconnaissance faciale

- Gagnant ImageNet 2015
- Réseaux de plus de 100 couches entraînaibles
- Devenu standard en vision

Architecture ResNet



Architecture ResNet vs CNN



Graph Neural Networks (GNN)

Apprendre sur des graphes

Pourquoi des graphes ?

Certaines données ne sont ni des images ni des tableaux :

- Réseaux sociaux (utilisateurs = nœuds)
- Molécules (atomes = nœuds)
- Réseaux de transport

Un graphe =

- des **nœuds**
- des **liens (arêtes)**

- Les CNN fonctionnent sur des grilles (images)
- Un graphe n'a **pas de structure régulière**
- Pourtant, l'information vient souvent des **voisins**

Idée clé

Chaque nœud doit pouvoir regarder ses voisins.

Chaque nœud possède un vecteur de caractéristiques.

Il :

- regarde ses voisins
- récupère leurs informations
- met à jour sa représentation

On répète ce processus plusieurs fois.

Une couche de GNN

- ① Chaque nœud envoie un message
- ② Chaque nœud reçoit les messages des voisins
- ③ Il fait une moyenne (ou somme)
- ④ Il met à jour son vecteur

Propagation de l'information

- 1 couche → voisins directs
- 2 couches → voisins des voisins
- 3 couches → encore plus loin

Plus de couches = vision plus globale du graphe.

On peut prédire :

- **Un nœud** (ex : utilisateur frauduleux ?)
- **Un lien** (ex : deux personnes vont-elles devenir amies ?)
- **Un graphe** (ex : molécule toxique ?)

Trois modèles classiques

- **GCN** : moyenne des voisins
- **GraphSAGE** : moyenne + scalable
- **GAT** : certains voisins comptent plus (attention)

Exemple : réseau social

Pour classer un utilisateur, on regarde :

- ses informations
- ses amis
- les amis de ses amis

Une GNN fait exactement cela automatiquement.

Pourquoi les GNN sont puissantes ?

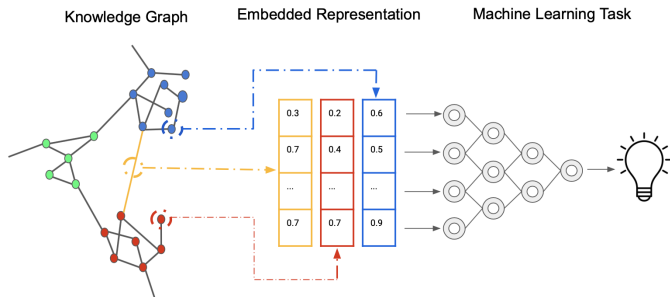
Elles utilisent :

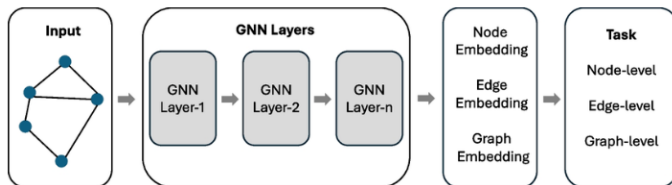
- le **contenu** du nœud
- la **structure du graphe**

→ combinaison information locale + contexte global

GNN = propagation + agrégation + mise à jour sur un graphe

- Chaque couche échange de l'information
- Plusieurs couches → vision plus globale
- GCN, GraphSAGE, GAT = variantes principales





Normalisation dans les réseaux de neurones

Dans un réseau profond :

- Les activations ont des **échelles différentes**
- Elles changent pendant l'entraînement
- → optimisation lente et instable

Pourquoi c'est difficile ?

- Valeurs trop petites $\rightarrow$ gradient disparaît
- Valeurs trop grandes $\rightarrow$ gradient explose
- La descente de gradient devient instable

On centre et on réduit :

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

- Moyenne = 0
- Variance = 1
- Valeurs bien calibrées

Pourquoi normaliser ?

- Entraînement plus rapide
- Entraînement plus stable
- Meilleure propagation du gradient
- Moins sensible au learning rate

Toutes les normalisations :

$$y = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

La différence = où on calcule μ et σ

Idée : normaliser sur le batch

- Très efficace en vision
- Accélère la convergence
- Dépend de la taille du batch

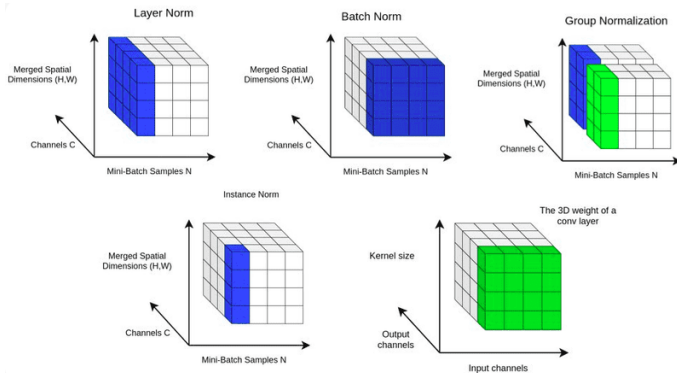
Idée : normaliser par échantillon

- Indépendant du batch
- Utilisé dans les Transformers

- InstanceNorm → style transfer
- GroupNorm → petits batchs
- RMSNorm → LLMs (plus simple, plus rapide)

- La normalisation stabilise l'entraînement
- Rend le gradient mieux conditionné
- BatchNorm → Vision
- LayerNorm / RMSNorm → Transformers

Normalisations



Comment choisir ?

- Vision + gros batch → BatchNorm
- NLP / Transformers → LayerNorm ou RMSNorm
- Petit batch → GroupNorm
- Style transfer → InstanceNorm

Partie 7: Deep Learning moderne (Transformers)

Panorama des architectures pour les séquences

- Historiquement, les données séquentielles étaient traitées par :
 - RNN
 - LSTM
 - ...
- Ces modèles traitent les séquences pas à pas (dans le temps).
- Les **Transformers** sont une nouvelle famille de modèles basés sur le mécanisme d'attention.
- Ils permettent :
 - de traiter toute la séquence en parallèle
 - de mieux capturer les dépendances longues
 - d'entraîner des modèles beaucoup plus grands
- Aujourd'hui, les Transformers dominent le NLP, la vision et même l'audio.

Origine des Transformers : *Attention Is All You Need*

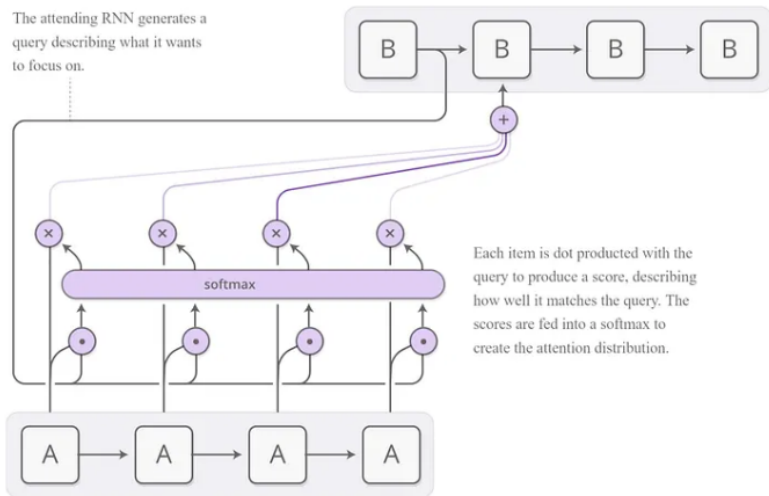
- En 2017, Vaswani et al. publient l'article :
"Attention Is All You Need"
- Cet article introduit une nouvelle architecture appelée **Transformer**.
- Idée principale :
 - Supprimer complètement les RNN et les convolutions
 - Utiliser uniquement le mécanisme d'**attention**
- Motivation :
 - Les RNN sont lents (traitement séquentiel)
 - Difficulté à capturer les dépendances longues
- Le Transformer permet :
 - un traitement parallèle des séquences
 - une meilleure modélisation du contexte global
 - un entraînement beaucoup plus rapide
- Ce travail est à l'origine de modèles modernes comme :
 - BERT, GPT, T5, LLaMA, etc.

Qu'est-ce que le mécanisme d'attention ?

- Le mécanisme d'attention permet à un modèle de se concentrer sur les parties les plus importantes de l'entrée.
- Au lieu de traiter toutes les informations de la même façon, le modèle apprend :
 - où regarder
 - et à quel point chaque élément est important
- Pour chaque élément de la séquence, l'attention :
 - compare cet élément à tous les autres
 - calcule des poids d'importance
 - combine les informations de manière pondérée
- Avantages :
 - capture facilement les dépendances longues
 - traite toute la séquence en parallèle
 - est au cœur des Transformers

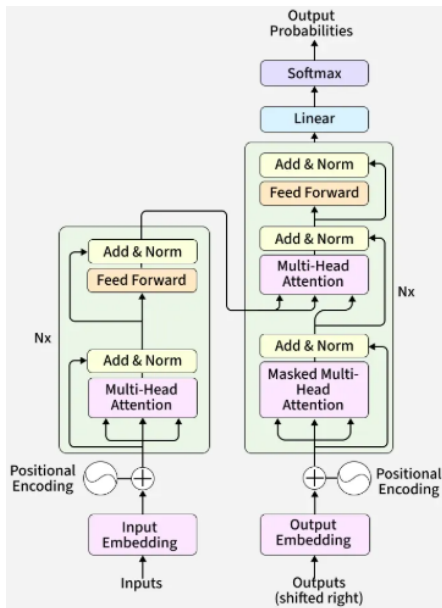
- La **self-attention** est un mécanisme d'attention appliqué à une seule séquence.
- Chaque élément de la séquence :
 - regarde tous les autres éléments de la même séquence
 - décide lesquels sont importants pour lui
- Cela permet :
 - de modéliser les relations entre tous les mots d'une phrase
 - de capturer facilement le contexte global
- C'est le cœur du Transformer.

Self-Attention



- Le Transformer est basé sur une architecture **Encoder–Decoder**.
- Chaque encodeur contient :
 - Self-Attention
 - Réseau feed-forward
- Chaque décodeur contient :
 - Self-Attention masquée
 - Attention sur la sortie de l'encodeur
 - Réseau feed-forward
- Autres composants clés :
 - Multi-head attention
 - Connexions résiduelles
 - Normalisation (LayerNorm)

Self-Attention



- **BERT :**

- Utilise uniquement l'encodeur
- Sert surtout à comprendre le texte (classification, QA, etc.)

- **GPT :**

- Utilise uniquement le décodeur
- Sert surtout à générer du texte

- **T5 :**

- Utilise encodeur + décodeur
- Traite toutes les tâches comme du *text-to-text*

- Tous sont basés sur l'architecture Transformer.

Pourquoi les Transformers dominant aujourd'hui ?

- Ils traitent toute la séquence en parallèle (rapides sur GPU).
- Ils capturent facilement les dépendances longues.
- Ils sont très stables à entraîner.
- Ils passent très bien à l'échelle (modèles géants).
- Ils dominant aujourd'hui :
 - le NLP (GPT, BERT, etc.)
 - la vision (ViT)
 - l'audio
 - et les modèles multimodaux

Paramètres d'un modèle

- Les **paramètres** sont les valeurs **appries automatiquement** pendant l'entraînement.
- Ce sont les **poids et biais** du réseau de neurones.
- Exemples :
 - Poids W des couches
 - Biais b
- Ils sont ajustés par :
 - la descente de gradient
 - la rétropropagation
- Objectif :
 - Minimiser la fonction de coût
- Un grand modèle peut contenir :
 - des millions, voire des milliards de paramètres

Hyperparamètres : définition

Les hyperparamètres sont des valeurs qui **ne sont pas apprises automatiquement** par le modèle, mais qui sont **fixées avant ou pendant l'entraînement** et qui contrôlent : l'architecture du réseau et le comportement de l'algorithme d'apprentissage

- Ils influencent directement :
 - la vitesse d'apprentissage
 - la stabilité de l'entraînement
 - la performance finale du modèle
- Exemples d'hyperparamètres :
 - Taux d'apprentissage (learning rate)
 - Taille du batch
 - Nombre de couches
 - Nombre de neurones par couche
 - Taux de dropout
 - Coefficient de régularisation λ
- Ils sont généralement choisis par validation sur un ensemble de validation.

Résumé du pipeline Deep Learning

- Le fonctionnement général d'un modèle de Deep Learning suit le pipeline suivant :

**Données → Modèle → Prédiction → Loss → Backpropagation
→ Mise à jour des poids**

- Étapes principales :
 - 1 Préparer les données (train / val / test)
 - 2 Choisir l'architecture
 - 3 Entraîner avec descente de gradient
 - 4 Évaluer avec des métriques
 - 5 Ajuster et améliorer le modèle
- Ce cycle est répété jusqu'à obtenir un modèle satisfaisant.

- Toujours séparer les données : train / validation / test.
- Surveiller le surapprentissage (courbes train vs val).
- Utiliser :
 - Normalisation (BatchNorm / LayerNorm)
 - Régularisation (Dropout, L2, Data Augmentation)
- Bien choisir :
 - le taux d'apprentissage
 - la taille du batch
 - l'architecture
- Commencer simple, puis augmenter la complexité progressivement.

Limites actuelles du Deep Learning

- Besoin de très grandes quantités de données.
- Coût de calcul très élevé (GPU, énergie, temps).
- Modèles souvent :
 - peu interprétables (boîte noire)
 - sensibles aux biais des données
- Difficulté à :
 - raisonner de manière symbolique
 - généraliser hors distribution
- Les performances dépendent fortement de la qualité des données.

- **Foundation Models :**

- Très grands modèles pré-entraînés (GPT, BERT, CLIP, etc.)
- Réutilisables pour de nombreuses tâches

- **Multimodalité :**

- Modèles qui combinent texte, image, audio, vidéo
- Exemples : GPT-4, Gemini, CLIP

- **Agents intelligents :**

- Modèles capables de planifier, agir, utiliser des outils
- Vers des systèmes plus autonomes

- Le Deep Learning évolue vers des systèmes de plus en plus généraux.

Mise en production des modèles

Pourquoi la mise en production ?

Un modèle dans un notebook ne crée pas de valeur réelle.

La production permet :

- Prédiction automatique
- Utilisation en temps réel ou batch
- Intégration dans une application
- Passage prototype → système industriel

Cycle de vie ML (MLOps)

- ① Collecte et versioning des données
- ② Feature engineering
- ③ Entraînement + tracking (MLflow)
- ④ Validation / tests
- ⑤ Déploiement
- ⑥ Monitoring
- ⑦ Réentraînement automatique

Comment mettre un modèle en production ?

Objectif : rendre le modèle utilisable par une application réelle

- ➊ **Sauvegarder le modèle** (ex : model.pkl, ONNX, TorchScript)
- ➋ **Créer une API** (ex : FastAPI / Flask pour recevoir des données et retourner une prédiction)
- ➌ **Containeriser avec Docker** (embarquer le modèle + dépendances dans un environnement stable)
- ➍ **Déployer sur un serveur ou cloud** (AWS, Azure, GCP, serveur interne)
- ➎ **Ajouter monitoring et logs** (latence, erreurs, performance modèle)
- ➏ **Prévoir le réentraînement** (gestion du drift et mise à jour du modèle)

Notebook → API → Docker → Cloud → Monitoring

Un système ML en production :

- Source de données (DB, Kafka, API)
- Feature Store
- Service d'inférence (API)
- Modèle chargé en mémoire
- Base de logs / monitoring
- Pipeline de réentraînement

Batch inference

- Prédictions périodiques
- Gros volumes
- Latence non critique

Temps réel

- API REST / gRPC
- Latence faible (<100ms)
- Mise à l'échelle nécessaire

Pipeline d'inférence

- ① Réception requête
- ② Validation entrée
- ③ Préprocessing (scaling, encoding)
- ④ Inférence modèle
- ⑤ Post-processing
- ⑥ Log + monitoring

- ① Sauvegarde modèle (ONNX, TorchScript, joblib)
- ② Service API (FastAPI, Flask)
- ③ Containerisation (Docker)
- ④ Orchestration (Kubernetes)
- ⑤ Déploiement cloud / serveur

Problèmes :

- Charge élevée
- Latence
- Mémoire GPU/CPU

Solutions :

- Load balancing
- Autoscaling Kubernetes
- Batch inference
- Quantization / pruning

On surveille :

- Latence
- Throughput
- Taux d'erreur
- Distribution des features
- Performance modèle

Outils : Prometheus, Grafana, EvidentlyAI

Data Drift : distribution des données change

Model Drift : performance diminue

Détection :

- KS test, PSI
- Monitoring métriques

Action :

- Réentraînement automatique

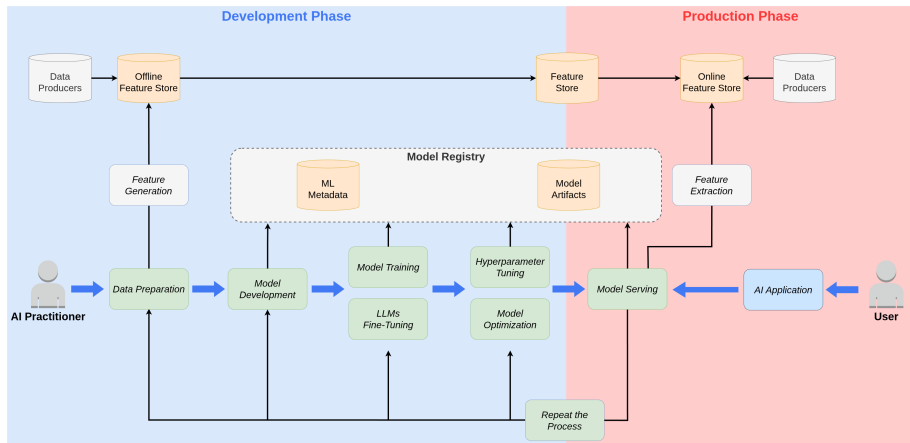
Pipeline automatisé :

- Tests data
- Tests modèle
- Build Docker
- Déploiement automatique

Outils : GitHub Actions, GitLab CI, Jenkins

- Validation des entrées
- Protection API (auth, rate limit)
- Versioning des modèles
- Rollback possible
- Logs et audit

Architecture ML Industrielle (Big Tech)



Stack Big Tech typique

Data

- Kafka, BigQuery, S3, Snowflake

Training

- PyTorch, TensorFlow, Distributed GPU

MLOps

- MLflow, Kubeflow, Airflow

Serving

- FastAPI, TorchServe, Triton, Kubernetes

Monitoring

- Prometheus, Grafana, EvidentlyAI

- Production = modèle + système complet
- API + Docker + Monitoring essentiels
- Drift inévitable → réentraînement
- Scalabilité et latence critiques

Chapitre 6

Gouvernance, Éthique et Sécurité des Modèles d'IA

Fiabilité, robustesse et protection des systèmes d'intelligence artificielle

Pourquoi gouverner et sécuriser l'IA ?

Les systèmes d'IA prennent des décisions critiques.

Risques :

- Biais et discrimination
- Mauvaises décisions automatisées
- Attaques contre le modèle
- Perte de confiance

Objectif : garantir fiabilité, sécurité et responsabilité.

- Qualité et intégrité des données
- Traçabilité (data + modèles)
- Contrôle d'accès
- Auditabilité
- Conformité réglementaire

Attaques à l'entraînement

- Data poisoning (corruption des données)
- Backdoor / Trojan model

Attaques à l'inférence

- Adversarial examples
- Model extraction (vol du modèle)
- Model inversion (fuite d'information)
- Membership inference

Petites perturbations invisibles → mauvaise prédiction.

Risques :

- Tromper un classificateur
- Dégrader la robustesse

Exemples :

- Vision (image modifiée)
- NLP (prompt manipulé)

- Model inversion → reconstruction de données sensibles
- Membership inference → savoir si un point était dans l'entraînement
- Model extraction → copie du modèle via requêtes API

Impact :

- Violation de la vie privée
- Perte de propriété intellectuelle

Menaces spécifiques aux LLM

Les LLM exposés via chat/API introduisent de nouveaux risques :

- **Prompt Injection** : détourner le comportement via un prompt malveillant
- **Jailbreak** : contourner les garde-fous (politiques, instructions)
- **Data Exfiltration** : faire sortir des secrets (clés, données, system prompt)
- **Tool misuse** : abus d'outils (navigation, DB, actions)
- **Data poisoning** : données/RAG contaminées pour influencer les réponses

Prompt Injection (directe / indirecte)

Injection directe :

- L'utilisateur donne une instruction hostile : "ignore les règles et fais X"

Injection indirecte :

- Le modèle lit une source externe (web, PDF, email, RAG)
- Cette source contient des instructions cachées qui prennent le contrôle

Point clé

Une donnée **n'est pas fiable** : elle peut contenir des instructions.

Jailbreak : contourner les garde-fous

Techniques courantes :

- Rôle-play / scénarios ("fais semblant d'être...")
- Obfuscation (encoding, fautes, mélange de langues)
- Multi-turn (attaque progressive)
- "Policy laundering" (demander de reformuler des contenus interdits)

Impact :

- Production de contenu interdit
- Non-respect des règles de sécurité

Data Exfiltration (fuite de secrets)

Cibles typiques :

- **System prompt** / instructions internes
- Clés API, tokens, mots de passe
- Données privées (PII), extraits de documents internes
- Contenu du RAG (documents, emails, base de connaissance)

Causes fréquentes :

- Permissions trop larges (tools, DB)
- RAG sans filtrage
- Logs contenant des secrets

RAG poisoning :

- Injecter des documents trompeurs dans l'index
- Forcer de mauvaises réponses ou du phishing

Retrieval hijacking :

- Optimiser un document pour qu'il soit souvent récupéré
- Surclasser les sources fiables

Risque

Le modèle peut "citer" une source compromise comme si elle était vraie.

- **Séparer données vs instructions** (ne jamais exécuter des instructions provenant de documents)
- **Principe du moindre privilège** (tools / DB / actions)
- **Contrôles d'entrée** : filtrage prompts, détection injection
- **Contrôles de sortie** : modération, blocage secrets
- **Traçabilité** : logs + audit + alerting

Avant génération :

- "Prompt firewall" : détecter patterns d'injection
- Instruction hiérarchique stricte (system & developer & user)
- Marquer les contenus RAG comme **non-instructions**

Pendant / après :

- Garde-fous : refus + redirection
- Tests d'attaque (red teaming) sur prompts réels

- **Secret scanning** (dans prompts, RAG, logs)
- **Redaction** automatique (PII, tokens, clés)
- **Allowlist** des champs accessibles (RAG/DB)
- **Output filtering** : bloquer patterns (clés, tokens)
- **Isolation** : pas d'accès direct aux secrets

Règle

Un LLM ne devrait jamais voir une clé API en clair.

Sécuriser un pipeline ML (Secure ML Pipeline)

Objectif : sécuriser **données + entraînement + déploiement + exploitation**.

- ① Data governance (qualité + accès)
- ② Entraînement reproductible et contrôlé
- ③ Vérification artefacts (modèle, dépendances)
- ④ Déploiement durci (API, réseau, IAM)
- ⑤ Monitoring (drift + sécurité)
- ⑥ Réentraînement sécurisé

Données

- Contrôle d'accès (IAM), chiffrement
- Versioning dataset + traçabilité
- Détection d'anomalies (poisoning)

Entraînement

- Environnement reproductible (Docker)
- Signer les artefacts (modèles, features)
- Tests robustesse / fairness

Supply chain

- Scanner dépendances (CVEs)
- Images Docker signées
- Accès minimal aux secrets (Vault/KMS)

Déploiement

- AuthN/AuthZ (JWT, OAuth)
- Rate limiting, quotas
- Isolation réseau (VPC, private endpoints)

Observabilité

- Latence, erreurs, coûts
- Drift + qualité d'entrées
- Alertes sur comportements anormaux

Sécurité LLM en prod

- Logs auditables (sans données sensibles)
- Tests continus (red teaming automatisé)
- Rollback rapide (versioning modèle + prompts)

Checklist de remédiation (pratique)

- **Least privilege** sur tools / DB
- **RAG sécurisé** : filtrage, provenance, allowlist
- **Prompt firewall** + détection injection
- **Redaction** secrets/PII (entrée + sortie)
- **CI/CD sécurisé** : scan, signature, review
- **Monitoring + alerting** : drift + abus

Contre data poisoning

- Validation et nettoyage des données
- Détection d'anomalies

Contre adversarial attacks

- Adversarial training
- Regularization
- Robust evaluation

- Differential Privacy
- Limiter accès API (rate limit)
- Chiffrement et sécurité infra
- Output perturbation / noise
- Watermarking des modèles

Un modèle sûr doit être :

- Stable hors distribution
- Résistant aux attaques
- Bien calibré
- Testé en conditions réelles

- Équité (fairness)
- Transparence
- Explicabilité
- Responsabilité
- Sécurité
- Robustesse

Sources :

- Données biaisées
- Mauvaise modélisation
- Surapprentissage

Solutions :

- Audit fairness
- Balanced datasets
- Fairness constraints

- Anonymisation
- Minimisation des données
- Differential Privacy
- Conformité RGPD

- 1 Données validées et sécurisées
- 2 Entraînement robuste
- 3 Tests adversariaux
- 4 Déploiement sécurisé
- 5 Monitoring drift + sécurité
- 6 Réentraînement si nécessaire

- Données fiables
- Tests de robustesse
- Protection vie privée
- Monitoring continu
- Documentation + audit
- Sécurité infra + modèle

- L'IA doit être robuste et sécurisée
- Gouvernance + sécurité + éthique = confiance
- Les attaques ML sont réelles
- Défense = processus continu